

Big M method

```
import numpy as np
import re

get_int = lambda: int(input())
get_ints = lambda: list(map(int, input().split()))
get_floats = lambda: list(map(float, input().split()))
get_str = lambda: input().strip()

EPS = 1e-9
M = 1e9

n = int(input("number of variables: "))
mode = input("minimise or maximise (min/max): ")

sign = -1 if "max" in mode else 1

print("objective coefficients:")
obj = get_floats()

m = int(input("number of constraints: "))

print("constraints (coeffs relation rhs per row):")

coef = []
rhs = []
rel = []

for i in range(m):
```

```
line = get_str()
line = line.replace("≥", ">=").replace("≤", "<=")

parts = re.split(r"(<|=|>|=|<|>)", line)

coef.append([float(x) for x in parts[0].split()])
rel.append(parts[1])
rhs.append(float(parts[2]))
```

```
cols = n
```

```
slack = {}
surplus = {}
artificial = {}
```

```
for i in range(m):
```

```
    if rel[i] == "<=":
        slack[i] = cols
        cols += 1
```

```
    elif rel[i] == ">=":
        surplus[i] = cols
        cols += 1
```

```
    artificial[i] = cols
    cols += 1
```

```
else:
    artificial[i] = cols
    cols += 1
```

```

table = np.zeros((m, cols))

for i in range(m):

    table[i, :n] = coef[i]

    if rel[i] == "<=":
        table[i, slack[i]] = 1

    elif rel[i] == ">=":
        table[i, surplus[i]] = -1
        table[i, artificial[i]] = 1

    elif rel[i] == "=":
        table[i, artificial[i]] = 1

b = np.array(rhs, dtype=float)

cost = [0.0] * cols

for j in range(n):
    cost[j] = obj[j] * sign

for j in artificial.values():
    cost[j] = M

basis = []

for i in range(m):

```

```

if rel[i] == "<=":
    basis.append(slack[i])
else:
    basis.append(artificial[i])

for _ in range(1000):

    B = table[:, basis]

    x = np.linalg.solve(B, b)

    y = np.linalg.solve(
        B.T,
        [cost[j] for j in basis]
    )

    reduced = []

    for j in range(cols):
        reduced.append(cost[j] - table[:, j] @ y)

    enter = None

    for j in range(cols):
        if j not in basis and reduced[j] < -EPS:
            enter = j
            break

```

```

if enter is None:

    ans = np.zeros(cols)

    for i, j in enumerate(basis):
        ans[j] = x[i]

    break

d = np.linalg.solve(B, table[:, enter])

if max(d) <= EPS:
    print("The problem is unbounded.")
    quit()

leave = -1
best = float("inf")

for i in range(m):

    if d[i] > EPS:

        ratio = x[i] / d[i]

        if ratio < best:
            best = ratio
            leave = i

basis[leave] = enter

```

```

possible = True

if "ans" in locals():

    for j in artificial.values():

        if ans[j] > EPS:
            possible = False

if "ans" not in locals():

    print("The problem is unbounded.")

elif not possible:

    print("The problem is infeasible.")

else:

    print("Optimal solution found.")

    for i in range(n):
        print("x%d = %.4f" % (i + 1, ans[i]))

    value = sum(
        obj[i] * ans[i]
        for i in range(n)
    )

    print("Objective value: %.4f" % value)

```

Vogel and MODI Method

```
import numpy as np

ri = lambda: int(input())
rl = lambda: list(map(int, input().split()))
rlf = lambda: list(map(float, input().split()))
rm = lambda: map(int, input().split())
rs = lambda: input().strip()

rows = int(input("Rows: "))
cols = int(input("Cols: "))

print("Cost Matrix:")
cost = [rlf() for _ in range(rows)]

print("Supply Vector:")
supply = rlf()

print("Demand Vector:")
demand = rlf()

total_supply = sum(supply)
total_demand = sum(demand)

if total_supply > total_demand:
    print(1)

    for i in range(rows):
```

```

        cost[i].append(1e5)

    demand.append(total_supply - total_demand)
    cols += 1

elif total_demand > total_supply:
    print(2)

    dummy_row = [1e5] * cols

    supply.append(total_demand - total_supply)
    cost.append(dummy_row)

    rows += 1

total = total_supply = total_demand

cost = np.array(cost)

answer = 0

table = np.zeros((rows, cols))
original_cost = cost.copy()

# ===== VAM =====

def calculate_penalties(cost):
    row_penalties = []
    col_penalties = []

```

```
for i in range(rows):
    min1 = 1000
    min2 = 1000

    for value in cost[i]:
        if value < min1:
            min2 = min1
            min1 = value
        elif value < min2:
            min2 = value

    row_penalties.append(min2 - min1)

for j in range(cols):
    min1 = 1000
    min2 = 1000

    for i in range(rows):
        value = cost[i][j]

        if value < min1:
            min2 = min1
            min1 = value
        elif value < min2:
            min2 = value

    col_penalties.append(min2 - min1)

return row_penalties, col_penalties
```

```
while total > 0:

    row_penalties, col_penalties = calculate_penalties(cost)

    max_row_penalty = max(row_penalties)
    max_col_penalty = max(col_penalties)

    if max_row_penalty >= max_col_penalty:

        for i in range(rows):

            if row_penalties[i] != max_row_penalty:
                continue

            minimum_cost = min(cost[i])

            for j in range(cols):

                if cost[i][j] != minimum_cost:
                    continue

                allocation = min(supply[i], demand[j])

                table[i][j] = allocation

                supply[i] -= allocation
                demand[j] -= allocation

                answer += allocation * cost[i][j]
```

```
|         if demand[j] == 0:
|             for k in range(rows):
|                 cost[k][j] = 1000
|         else:
|             for k in range(cols):
|                 cost[i][k] = 1000
|
|         break
|
|     break
|
| else:
|
|     for j in range(cols):
|
|         if col_penalties[j] != max_col_penalty:
|             continue
|
|         minimum_cost = min(
|             cost[i][j]
|             for i in range(rows)
|         )
|
|         for i in range(rows):
|
|             if cost[i][j] != minimum_cost:
|                 continue
|
|             allocation = min(supply[i], demand[j])
|
|             table[i][j] = allocation
```

```

    supply[i] -= allocation
    demand[j] -= allocation

    answer += allocation * cost[i][j]

    if demand[j] == 0:
        for k in range(rows):
            cost[k][j] = 1000
    else:
        for k in range(cols):
            cost[i][k] = 1000

    break

    break

if max_row_penalty == 0.0 and max_col_penalty == 0.0:
    break

total -= table[i][j]

print(table)
print("Cost:", answer)

```

```
# ===== MODI =====
```

```
def modi_iteration():
```

```
    seen_u = [0 for _ in range(rows)]
    seen_v = [0 for _ in range(cols)]
```

```
    u = [0 for _ in range(rows)]
    v = [0 for _ in range(cols)]
```

```
    max_allocations = 0
    starting_row = 0
```

```
    for i in range(rows):
```

```
        | current_allocations = sum(
        |     1
        |     for j in range(cols)
        |     if table[i][j] > 0
        | )
```

```
        | if current_allocations > max_allocations:
        |     max_allocations = current_allocations
        |     starting_row = i
```

```
def calculate_uv(i, j, is_row):
```

```
    | if is_row:
```

```
        | for column in range(cols):
```

```

        if seen_v[column] == 0 and table[i][column] > 0:

            seen_v[column] = 1
            v[column] = original_cost[i][column] - u[i]

            calculate_uv(i, column, False)

    else:

        for row in range(rows):

            if seen_u[row] == 0 and table[row][j] > 0:

                seen_u[row] = 1
                u[row] = original_cost[row][j] - v[j]

                calculate_uv(row, j, True)

seen_u[starting_row] = 1

calculate_uv(starting_row, 0, True)

maximum_penalty = 0
selected_row = -1
selected_col = -1

for i in range(rows):

    for j in range(cols):

```

```

        if table[i][j] == 0:

            penalty = (
                u[i]
                + v[j]
                - original_cost[i][j]
            )

            if penalty > maximum_penalty:
                maximum_penalty = penalty
                selected_row = i
                selected_col = j

    if maximum_penalty <= 0.001:
        return False

    nodes = []

    for i in range(rows):

        for j in range(cols):

            if table[i][j] > 0:
                nodes.append((i, j))

    nodes.append((selected_row, selected_col))

    def find_loop(row, col, path, horizontal):

        if len(path) > 3 and (row, col) == (

```

```

        selected_row,
        selected_col
    ):
        return path

for next_row, next_col in nodes:

    if (
        (next_row, next_col) in path
        and (next_row, next_col)
        != (selected_row, selected_col)
    ):
        continue

    if horizontal and next_row == row and next_col != col:

        result = find_loop(
            next_row,
            next_col,
            path + [(next_row, next_col)],
            False
        )

        if result:
            return result

    elif not horizontal and next_col == col and next_row != row:

        result = find_loop(
            next_row,
            next_col,

```

```
        path + [(next_row, next_col)],
        True
    )

    if result:
        return result

    return None

loop_path = (
    find_loop(
        selected_row,
        selected_col,
        [(selected_row, selected_col)],
        True
    )
    or
    find_loop(
        selected_row,
        selected_col,
        [(selected_row, selected_col)],
        False
    )
)

if not loop_path:
    return False

loop_path = loop_path[:-1]

minus_cells = [
```

```
    loop_path[k]
    for k in range(1, len(loop_path), 2)
]
```

```
theta = min(
    table[row][col]
    for row, col in minus_cells
)
```

```
for k in range(len(loop_path)):
```

```
    row, col = loop_path[k]
```

```
    if k % 2 == 0:
        table[row][col] += theta
    else:
        table[row][col] -= theta
```

```
return True
```

```
while modi_iteration():
    pass
```

```
print("\nFinal:")
print(table)
```

```
answer = 0
```

```
for i in range(rows):
```

```
    for j in range(cols):
        answer += table[i][j] * original_cost[i][j]
```

```
print("Cost:", answer)
```